Test Integration HLD

Author(s)

Name	Email
Raja Nand Sharma	rsharma4@infoblox.com

Designated Reviewers

[Note that this document should be reviewed by one or more Solution/Domain Architects as well as an engineering manager or team lead before submitting it for ADAPT approval.]

Name	Role	Sign-off Date
Adam Casella	Solution Architect	
Narasimha Reddy	Engineering Manager	

ADAPT Review(s)

Date	Outcome	Link

Revision History

Revision	Date	Description
0.1	Jun 6, 2025	Initial draft
	Jun 12, 2025	Added API for fetching the log data parameters

Contents

Introduction 3

Assumptions 4

Requirements 4

Service Architecture 5

Operation 5

Implementation 6
- Authentication 6
- Authorization 7
- Ingress 7
- Audit 8
- Logging 8
- Alerting 8
- Performance 9
- Security 9
- External Resources 9
- Third-Party Software 9
- Metrics/Telemetry 9
- Rate Limits 9
- UX/UI 9
- High Availability/Resiliency 10
- Scalability 10
- Backup 10
- Data Lake 10
- Disaster Recovery 10
- Data Retention 10

Data Model 11

API 11
- AddWidget() 11
  - Parameters 11
  - Roles allowed 11
  - Returns 11
- CheckoutWidget() 12
  - Parameters 12
  - Roles allowed 12
  - Returns 12
- ListWidgets() 12
  - Parameters 12

Internal

Roles allowed 12
Returns 12

Testing 12

Migration 12

Deployment Architecture 13

Upgrades 13

Service Dependencies 13

Service Killers 13

Data Analytics 13

References 14

Introduction

This document describes the Test Integration feature for the CDC (Change Data Capture) ecosystem. This feature enables customers to validate their integration configurations by sending test events through their configured data flows, ensuring proper connectivity, authentication, and data transformation before production use.
The initial implementation focuses on flows with Infoblox Cloud as the source and SOAR Light as the destination, with plans for expansion to other integration types in future releases.

Internal

Assumptions

The service assumes the following:

- **Scope Limitation:** Initial implementation supports only Infoblox Cloud → SOAR Light flows
- **Single Log Type Testing:** Test events are triggered for one log type at a time
- **Authentication:** Users have valid authentication tokens and appropriate permissions
- **Infrastructure:** All required infrastructure components (Kafka, PostgreSQL, etc.) are operational
- **Network Connectivity:** Reliable network connectivity exists between all components

Requirements

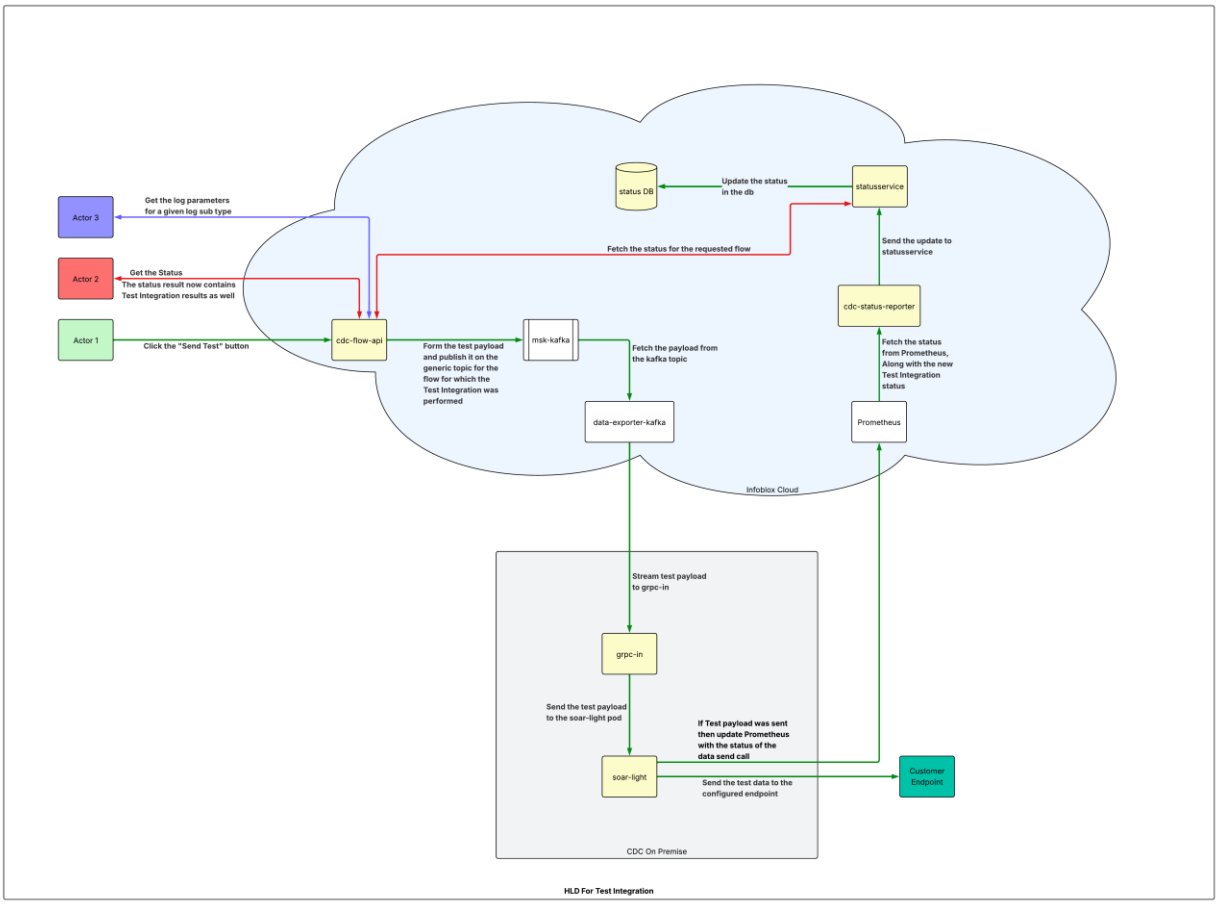
Customers currently lack a way to confirm that integrations are correctly configured and functioning, including connectivity, encryption, and configuration to ensure the integration will operate as expected when the right event is received.

V1 Story: PTCDC-2343 [FS] Capability to test integration

Please refer Jira Story.

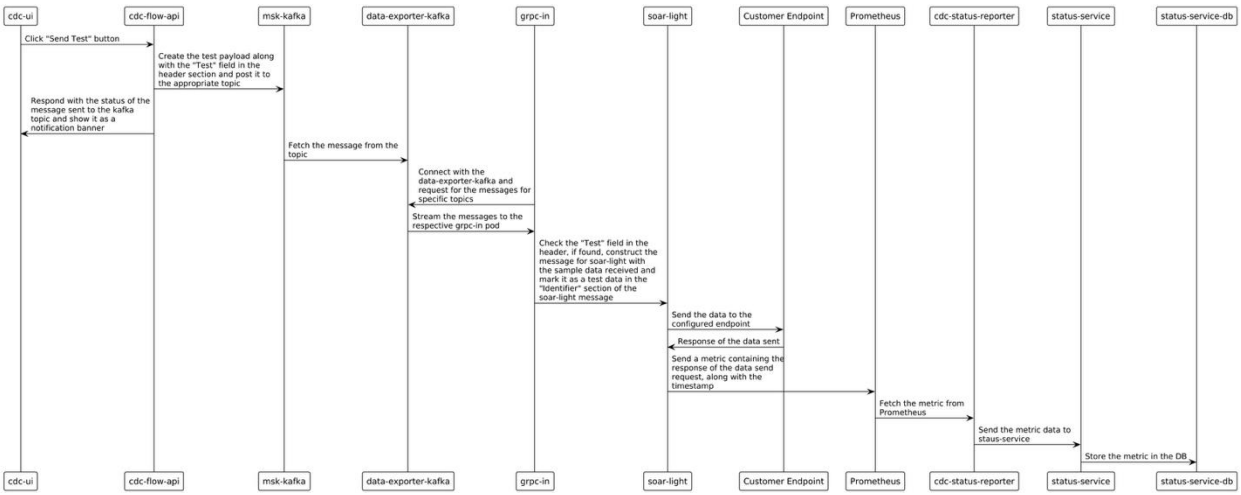
Internal

Service Architecture

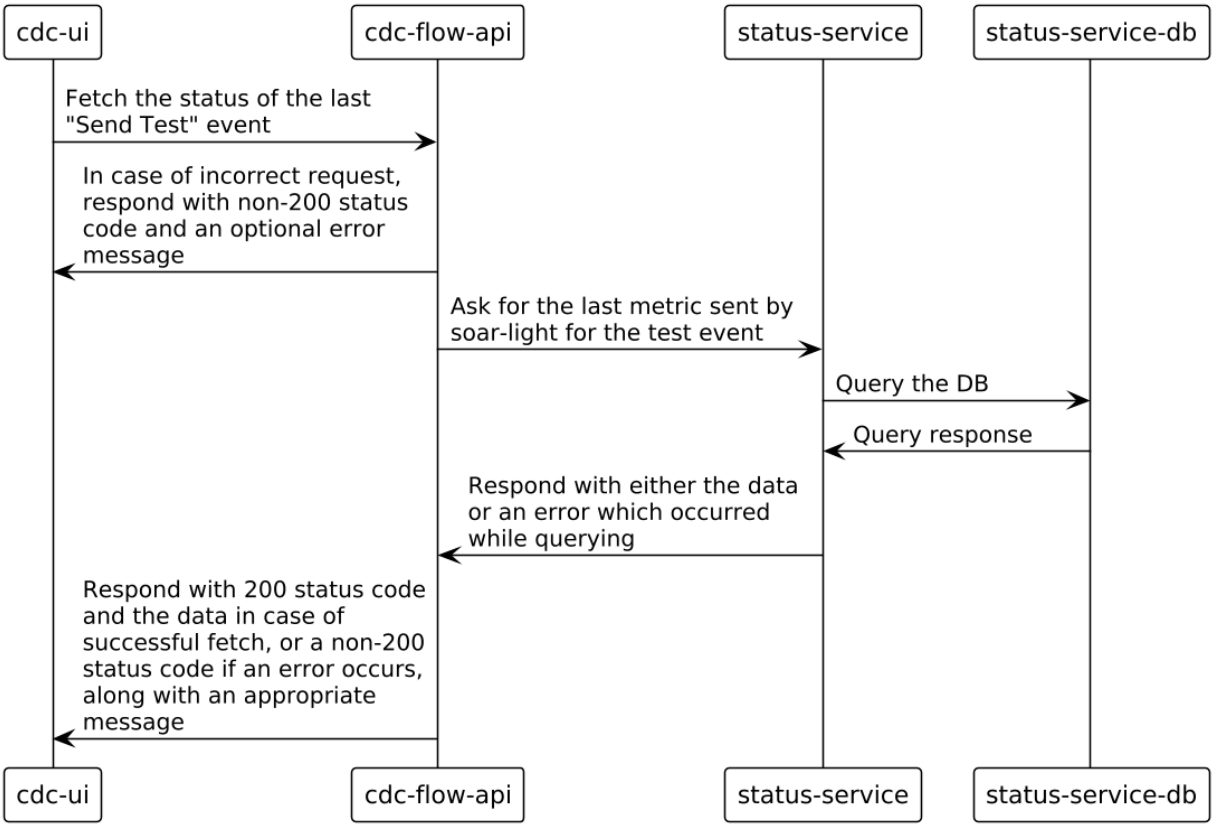


Link: https://lucid.app/lucidchart/8899289f-e426-44bd-84ac-9b71bffcb046/edit?viewport_loc=-2412%2C-1666%2C5598%2C3269%2C0_0&invitationId=inv_b890e3ef-8429-4501-a680-f35e94b3b14f

Operation



Sequence diagram for Test Integration action



Sequence diagram for fetching the last status of Test integration

Implementation

Authentication

No change in the current authentication model.

Internal

Authorization

No change in the current authorization model.

Entitlements

Customer should have the license to use Automation (soar-light)

Ingress

The ingress for the service is cdc-flow-api. The 3 new APIs will be under /test, namely

1. /v2/test_integration/trigger
    a. Method: POST
    b. Content-Type: application/json
    c. Sample request structure:
    ```
    {
        "flow_id": "12345",
        "source_log_subtype": "TD_QUERY",
        "data": {}        // Will change according to `log_subtype`
    }
    ```
    d. Sample response structure:
    ```
    {
        "message": "" // Will vary upon the status code
    }
    ```
    e. Actions performed:
        i. Construct the topic name for the `flow_id` sent
        ii. Construct the data in the format, grpc-in expects for the `log_subtype`
        iii. Post the data on the constructed topic (in step i.) on msk-kafka server
        iv. An erroneous or non-erroneous response is sent back, based on the data sent to the topic successfully or not.
        v. The status is shown on the UI in a notification banner
2. /v2/test_integration/log_parameters
    a. Method: GET
    b. Content-Type: application/json
    c. Sample request structure:
    ```
    {
        "source_log_subtype": ""
    }
    ```
    d. Sample response structure:
    ```
    {
        "log_fields": {
            "Rip": "dst",
    ```


Internal

```
        "Transport_protocol": "proto",
       ...
    },
    "editable_fields": [
        {
            "name": "Policy Action",
            "type": "string"
        },
        {
            "name": "Risk Score",
            "type": "number"
        },
        ...
    ]
}
```

3. /display/flows/{flow_id} (Modification)
    a. Method: GET
    b. Content-Type: application/json
    c. Sample request structure: Not changed
    d. Sample response structure:

```
flow: {
   ...,
   "test_event": {
      "log_name": "lalala log",
      "sent: Date,
      "status: string,
      "message": "log ka message"
       }
    }
```

Audit

No explicit auditing is required for the said APIs and actions

Logging

Standard logging practices will be used

Alerting

No alerts are required for this feature.

Internal

Performance

The same as other APIs in cdc-flow-api.
In addition to that, the /test/trigger API is not synchronous, and will return a status code once the message is posted on the msk-kafka server. The actual result of the action will eventually be available via the 2nd API (status API).
The 2nd API is already a part of cdc-flow-api and the same performance numbers will follow as per the current implementation.

Security

N/A

External Resources

No additional resources. Will use components from the current infrastructure, namely:

1. Kafka
2. PostgreSQL

Third-Party Software

N/A

Metrics/Telemetry

A new “test” label will be added for the feature in the existing metric sent to the Prometheus server. The label will have the following values:

1. Flow ID
2. Timestamp (of success or failure)
3. Source Log Type
4. Status
5. Status Message

Rate Limits

N/A

UX/UI

The feature requires multiple UI components to be added. The new additions to the UI are the dialog boxes for triggering the test and a modification in the status panel to show the status of

Internal

the test.
The wireframe is available here.

High Availability/Resiliency

Follows the same resiliency pattern as the existing APIs of cdc-flow-api and the following components:

1. Cloud
    a. Cdc-flow-api
    b. Msk-kafka server
    c. Data-exporter-kafka
    d. Prometheus server
    e. Cdc-status-server
    f. Status-service
    g. Status-service-db
2. On-Prem
    a. Grpc-in
    b. Soar-light

Scalability

N/A

Backup

The same as what status-service team follows for their database service.

Data Lake

N/A

Disaster Recovery

N/A

Data Retention

Each flow id will have one entry in the status-service DB; the same entry will be updated for each flow id.
The entry should be deleted once the corresponding flow is deleted.

Internal

Data Model

No addition or update to the schema is needed.

API

API endpoints must be designed with Infoblox guidelines and best practices in mind:

- REST API Syntax Specification

When Swagger documentation has been developed, link to it in this document, as the Swagger documentation will be the source of truth for the APIs.

TestTrigger()

The TestTrigger API will be the entry point to the test integration action.

Parameters

- flow_id: string
- source_log_subtype: string
- data: bytes

Roles allowed

- Accounts with Automation entitlement

Returns

- error: string // Optional: Only in case of 503

Status code

- 200 – Successful return
- 503 – If the message was not posted to the kafka topic

LogParameters()

The LogParameters API will provide the UI the fields that the log subtype supports.

Parameters

- source_log_subtype: string

Internal

Roles allowed

- Accounts with Automation entitlement

Returns

- all_fields: JSON (field name: field value pair)
- editable_fields: Array of Objects as defined below
    - name: string
    - type: string

The editable_fields.type will be used by the UI to validate the value given by the user for the field. For instance, if the type is set to “number” the value given by the user should be a number. Entering any other type of value will result in an error in the UI.

Status code

- 200 – Successful return
- 503 – If the message was not posted to the kafka topic

FlowStatus()

Refer above

Testing

[REQUIRED: Describe testing methodologies, including integration and functional tests, and any unusual unit tests.

OPTIONAL: Add details of SAAT (Support Analysis Automation Tool) support. For example, whether this design should include SAAT support (e.g., failure codes, strings, etc) that can be used as part of signature in a support bundle.]

Migration

The structure for the API call from cdc-status-reporter to status-service & cdc-flow-api to status-service will need to change if the API structure from the cdc-status-reporter changes.

Internal

Deployment Architecture

[OPTIONAL: Describe deployment architecture. This may be fairly simple if the standard Helm chart method is used. Note special circumstances like being deployed as a sidecar. Also, include any required external resources such as Kafka, RDS, Elasticsearch, etc., along with necessary versions]

Upgrades

N/A

Service Dependencies

At the time of writing, these services depend on this service:

- status-service

Service Killers

Service Name	Impact
cdc-flow-api	Full down
data-exporter-kafka msk-kafka	Unable to trigger the test action
Prometheus cdc-status-reporter status-service status-service DB	Unable to update the status of the flow

Data Analytics

N/A

Internal

References

N/A

Internal